

UBS Ticket Intelligence: Workflow, Grounding, and Value

Executive summary

This project turns a noisy operational ticket stream into explainable themes and evidence-backed root-cause candidates. The central design choice is a closed-world grounding contract: the system may use only the supplied ticket and change data, and it must send novel or weakly supported cases to a human instead of inventing an answer.

The deterministic code decides what is supported. An LLM, when added, explains only that verified evidence in natural language. It does not decide whether a case is known, search for external causes, or create an unsupported solution.

1. Project workflow

```
Tickets and changes
|
v
Load JSON and remove ground-truth fields
|
v
Normalize ticket text and match controlled rules
|
v
Group tickets into clusters
|
v
Check system, region, time, and text evidence against real changes
|
v
Calculate state, confidence, and root-cause status
|
+--> Known: prepare evidence-backed explanation
+--> Variant: human review
+--> Unknown: human expert, no cause claim
|
v
Optional LLM explanation using only the evidence pack
|
v
Validate every generated entity before display
```

The implemented pipeline is `scripts/groundedcluster.py`. It reads JSON tickets and changes, ignores every field whose name begins with `_`, and writes an auditable cluster result. The `gttheme` field is evaluation metadata only; it is never used to choose a cluster.

2. The rule book

The current practical rule book is the `RULES` tuple in `scripts/groundedcluster.py`. Each rule contains a name, allowed systems, keywords, and optional excluded terms:

Rule Systems Purpose

'mobile_login_failure' MobileApp, eBanking Login, PIN, biometric, 2FA, and update problems

'fx_settlement' FX-Trading FX, settlement, rates, feeds, and mismatch problems
'password_reset' eBanking Password, reset, credentials, and lockout problems
'statement_download' eBanking Statement export and PDF availability problems
'reporting_slow_month_end' Reporting Reporting, month-end, latency, and queue problems
'card_blocked_abroad' CardServices Card declines and travel-related blocks
'onboarding_upload' Onboarding Identity documents, upload, and validation problems
'payment_limits' Payments Beneficiary, payee, limit, and authorisation problems

The payment-limit rule excludes IBAN, Austrian, and beneficiary-validation terms. This prevents the intentionally novel IBAN scenario from being incorrectly absorbed into a known payment category.

3. Category matching and clustering

For each ticket and each rule, the script normalizes the title and description, finds approved terms, and calculates:

```
keyword_score = matched rule terms / total rule terms  
system_score = 1.0 if ticket.system is allowed, otherwise 0.0  
  
ticket_category_score =  
0.7 * keyword_score + 0.3 * system_score
```

The highest-scoring rule wins. If the best ticket score is below 0.20, the ticket is assigned to unclassified rather than forced into a known category. Tickets assigned to the same rule are grouped into one cluster. The cluster category score is the average of the ticket scores in that cluster.

4. Change evidence and root-cause candidates

A change is accepted as candidate evidence only when all checks pass:

1. The change system matches the cluster system.
2. At least one change region overlaps a cluster region.
3. The change happened before the first ticket.
4. The change happened within 7 days by default; this is the --change-window-days parameter.
5. At least one meaningful word from the change title appears in the ticket corpus.

Candidate changes are ranked by linked-ticket count, matching-term count, time proximity, and change ID. A candidate is evidence for investigation, not proof of causation. The output records the leading change in a rootcause object:

Status Meaning

'candidate' A real change passed all evidence filters
'not_established' No supplied change passed all evidence filters
'withheld_unknown' The cluster is novel, so a cause claim is forbidden

5. Evidence strength and confidence

Evidence strength is based on accepted candidate changes:

evidence_strength = min(1.0, accepted_candidate_changes / 2)

Therefore, zero changes gives 0.0, one gives 0.5, and two or more gives 1.0.

Confidence is deliberately conservative:

```
evidence_confidence = 0.5 + 0.5 * evidence_strength
confidence = min(cluster_category_score, evidence_confidence)
```

An unknown cluster is additionally capped at 0.30. This means good keyword similarity cannot create high confidence when supporting evidence is absent, and an unfamiliar case cannot enter an automated path merely because it sounds plausible.

6. Seen, variant, and unseen decisions

The cluster average determines the state:

Cluster category score	State	Route	Allowed conclusion
'>= 0.60'	'KNOWN'	'evidence_review'	Explain the pattern and approved evidence
'0.30-0.59'	'KNOWN-VARIANT'	'human_review'	Describe the pattern, but require review
'< 0.30'	'UNKNOWN'	'human_expert'	Describe only; no root-cause claim

For UNKNOWN, the output sets causeclaimallowed to false, sets the root-cause status to withheldunknown, and leaves changeid as null.

7. Where AI steps in

The current script is deterministic and does not call an LLM. That is intentional: it

creates a trustworthy evidence pack first. An LLM can then help with language and judgement, but only inside these boundaries:

- summarize the verified cluster;
- explain observed systems, regions, dates, symptoms, and severity;
- describe a ranked change candidate;
- select a solution from an approved playbook for a known category;
- state when evidence is insufficient.

The LLM must not use external knowledge, invent ticket or change IDs, name systems or

regions absent from the evidence, or create a solution for an unknown category. A later

output validator should check every ticket ID, change ID, system, region, date, quote, and cause against the evidence pack. Invalid output is suppressed and routed to a human.

8. Test data and results

The repository contains 1,401 tickets, 40 changes, and ten scenario datasets. The tests are in tests/grounding and run with:

```
python -m unittest discover -s tests/grounding -p 'test_*.py' -v
```

The latest suite result is:

Ran 10 tests
OK

Key demonstrations:

Scenario Observed behavior Value

Known login 'mobile_login_failure', 'KNOWN-VARIANT', real candidate such as

'CHG-0042' Finds a known pattern while retaining review controls

FX recurrence 'fx_settlement', 'KNOWN', no unsupported change candidate Shows that known does not mean causation is invented

Unknown IBAN 'unclassified', 'UNKNOWN', confidence ' ≤ 0.30 ', human expert Finds novelty and withholds the cause

Card abroad Known card cluster with system and region evidence Preserves geographic context

Singleton-heavy Six separate small clusters Avoids fabricating one large incident

The tests check ticket coverage, source-only IDs, no external sources, no ground-truth

use, confidence caps, correct routing, and safe root-cause behavior. They validate the

grounding contract; they are not a claim that clustering accuracy is 100 percent.

9. Project value

The value is controlled automation for operational risk teams:

- Speed: routine patterns can be grouped and explained consistently.
- Traceability: every claim can point back to source tickets and real changes.
- Risk control: unseen or weakly supported problems cannot silently become confident automated actions.
- Human focus: experts spend time on novel and high-risk cases instead of searching through repetitive tickets.
- Scalability: deterministic processing handles the bulk of tickets, while a future LLM can operate on clusters and evidence packs rather than every individual ticket.

The defensible claim is not that the system eliminates all hallucination. It is that the workflow establishes a controlled grounding boundary: the code decides what the data supports, the LLM explains only that support, and humans handle what remains uncertain.